



Vivekananda College of Engineering & Technology

[A Unit of Vivekananda Vidyavardhaka Sangha, Puttur ®]

Affiliated to Visvesvaraya Technological University

Approved by AICTE New Delhi & Govt of Karnataka

TCP02

Rev 1.4

CS

21/07/25

COURSE PLAN

A. COURSE OVERVIEW

Degree:	BE	Programme:	CS
Academic Year:	2025-26	Semester:	VII
Course Title:	Parallel Computing	Course Code:	BCS702
L-T-P-S:	3:0:2:0	Duration of SEE:	3 Hrs
Total Contact Hours:	40 hours Theory + 8-10 Lab slots	SEE Marks:	50 Marks*
CIE Marks:	50	IA Test	25
Credits:	4	Components	25
Lesson Plan Author:	Prof. Rashmi P.C	Sign: 	Date: 21.07.2025
Checked By:	Prof. Dinesh Prasanna	Sign: 	Date: 22.07.2025

*The SEE will be conducted for 100 marks and proportionally reduced to 50 marks.

B. PREREQUISITES

- Basics of Java Programming (BPLCK105C/205C)
- Analysis & Design of Algorithms (BCS401)
- Analysis & Design of Algorithms Lab (BCSL404)
- Computer Network (BCS502)
- Operating System (BCS303)

C. COURSE DESCRIPTION

i) Course Outcomes
At the end of the course, the student will be able to: 1. Explain the need for parallel programming 2. Demonstrate parallelism in MIMD system. 3. Apply MPI library to parallelize the code to solve the given problem. 4. Apply OpenMP pragma and directives to parallelize the code to solve the given problem. 5. Design a CUDA program for the given problem.
ii) Relevance of the Course
• Internet of Things (BCS701) • Cryptography & Network Security (BCS703) • Major Project Phase-II (BCS786)
iii) Applications areas
• Artificial Intelligence and Machine Learning • Cyber Security. • Data base interaction • Cloud Computing • IOT etc.



Vivekananda College of Engineering & Technology

[A Unit of Vivekananda Vidyavardhaka Sangha, Puttur ®]

Affiliated to Visvesvaraya Technological University

Approved by AICTE New Delhi & Govt of Karnataka

TCP02

Rev 1.4

CS

21/07/25

COURSE PLAN

Rash

Dinesh

Shree

Prepared by: Prof. Rashmi P.C

Checked by: Dinesh Prasanna

HOD

D1. ARTICULATION MATRIX, CO v/s PO

Mapping of CO to PO												
Cos	POs											
	1	2	3	4	5	6	7	8	9	10	11	12
1.Explain the need for parallel programming	3	2	-	-	-	-	-	-	-	-	-	-
2.Demonstrate parallelism in MIMD system.	3	2	2	-	-	-	-	-	-	-	-	-
3.Apply MPI library to parallelize the code to solve the given problem.	3	3	3	-	-	-	-	-	-	-	-	-
4. Apply OpenMP pragma and directives to parallelize the code to solve the given problem.	3	3	2	-	-	-	-	-	-	-	-	-
5.Design a CUDA program for the given problem.	3	3	3	-	-	-	-	-	-	-	-	-

Note: Mappings in the Tables D1 (above) and D2 (below) are done by entering in the corresponding cell the Correlation Levels in terms of numbers. For Slight (Low): 1, Moderate (Medium): 2, Substantial (High): 3 and for no correlation: “ - ”.

D2. ARTICULATION MATRIX, CO v/s PSO

Mapping of CO to PSO			
COs	PSOs		
	PSO1	PSO2	PSO3
1.Explain the need for parallel programming	-	2	2
2.Demonstrate parallelism in MIMD system.	-	2	2
3.Apply MPI library to parallelize the code to solve the given problem.	-	3	2
4. Apply OpenMP pragma and directives to parallelize the code to solve the given problem.	-	3	2
5.Design a CUDA program for the given problem.	-	2	2

E. MODULE PLANS

MODULE – I

Title:	Introduction to parallel programming, Parallel hardware and	Appr. Time:	8 Hrs
--------	---	-------------	-------



Vivekananda College of Engineering & Technology

[A Unit of Vivekananda Vidyavardhaka Sangha, Puttur ®]

Affiliated to Visvesvaraya Technological University

Approved by AICTE New Delhi & Govt of Karnataka

TCP02

Rev 1.4

CS

21/07/25

COURSE PLAN

parallel software		
MO:		Bloom's
At the end of the Module, the student will be able to:		Level
1. Understand the Fundamentals of Parallel Programming		L2
2. Identify Parallel Hardware and Software Architectures		L2
3. Analyse Interconnection Networks		L2
4. Differentiate Memory Architectures		L2
5. Understand Cache Coherence Mechanisms		L2
6. Coordinate Parallel Processes and Threads		L3
Lesson Schedule:		
Lecture No.	Portion to be covered	CO
1	Introduction: Classifications of parallel computers	CO1
2	SIMD systems, MIMD systems,	CO1
3	Interconnection networks,	CO1
4	Cache coherence	CO1
5	Shared-memory vs. distributed-memory	CO1
6	Coordinating the processes/threads	CO1
7	Shared-memory	CO1
8	Distributed-memory	CO1
Application Areas:		
• Supercomputing and High-Performance Computing (HPC) • Artificial Intelligence & Machine Learning • Cryptography and Cybersecurity • IOT		
Review Questions / Questions appeared in the previous year (CO):		
1	Explain the main classifications of parallel computers with suitable examples.(CO1)	
2	Describe interconnection networks. How do they affect the performance of parallel systems? (CO1)	
3	Explain cache coherence. Why is it important in shared-memory systems? (CO1)	
4	Compare and contrast shared-memory and distributed-memory architectures. (CO1)	
5	Discuss the role of parallel hardware and software in improving computational efficiency. (CO1)	
6	Describe the issues faced in distributed-memory systems and how message-passing helps to solve them. (CO1)	
7	List and explain any four types of interconnection topologies used in parallel computers.(CO1)	

MODULE – II

Title:	GPU programming, Programming hybrid systems, MIMD systems, GPUs, Performance	Appr. Time:	8 Hrs
MO:		Bloom's	
At the end of the Module, the student will be able to:		Level	
1.Understand GPU programming basics.		L2	
2. Learn programming techniques for hybrid systems (CPU + GPU).		L3	
3.Differentiate between MIMD systems and GPUs.		L3	
4.Develop simple GPU-based programs.		L2	



Vivekananda College of Engineering & Technology

[A Unit of Vivekananda Vidyavardhaka Sangha, Puttur @]

Affiliated to Visvesvaraya Technological University

Approved by AICTE New Delhi & Govt of Karnataka

TCP02

Rev 1.4

CS

21/07/25

COURSE PLAN

5. Analyze performance of parallel and hybrid systems. L2

6. Understand GPU programming basics. L3

7. Learn programming techniques for hybrid systems (CPU + GPU). L3

Lesson Schedule:

Lecture No.	Speedup and efficiency in MIMD systems,	CO
1	Amdahl's law.	CO2
2	Speedup and efficiency in MIMD systems.	CO2
3	Continuation.	CO2
4	Scalability in MIMD systems,	CO2
5	Taking timings of MIMD programs, GPU performance.	CO2
6	Continuation.	CO2
7	Continuation	CO2
8	GPU performance.	CO2

Application Areas:

Deep Learning and AI Model Training
Real-time Image and Video Processing
Big Data Analytics on GPU Clusters
Scientific Simulations and Modeling
Cryptography and Cybersecurity

Review Questions / Questions appeared in the previous year (CO):

1	What is GPU programming? How does it differ from CPU-based programming? (CO2)
2	Explain the architecture and working of MIMD systems. (CO2)
3	What are hybrid systems? Give examples of CPU-GPU combined architectures. (CO2)
4	List and explain applications where GPU computing offers better performance than CPU computing. (CO2)
5	Discuss the factors affecting the performance of GPU-based systems. (CO2)
6	How does parallelism in GPUs improve real-time processing tasks? (CO2)
7	Compare the performance benefits of MIMD systems and GPUs in parallel computing tasks. (CO2)

MODULE – III

Title:	Distributed memory programming with MPI:	Appr. Time:	8 Hrs
MO:			Bloom's Level
At the end of the Module, the student will be able to:			
1. Understand MPI functions			L2
2. Apply trapezoidal rule using MPI			L2
3.Handle I/O in MPI programs			L2
4. Use collective communication			L2
5. Work with MPI-derived datatypes			L2
6.Evaluate MPI program performance			L2
7.Implement parallel sorting algorithm			L2
8. Understand MPI functions			L2
9. Apply trapezoidal rule using MPI			L3



Vivekananda College of Engineering & Technology

[A Unit of Vivekananda Vidyavardhaka Sangha, Puttur @]

Affiliated to Visvesvaraya Technological University

Approved by AICTE New Delhi & Govt of Karnataka

TCP02

Rev 1.4

CS

21/07/25

COURSE PLAN

Lesson Schedule:

Lecture No.	Portion to be covered	CO3
1	MPI functions,	CO3
2	The trapezoidal rule in MPI	CO3
3	Dealing with I/O	CO3
4	Collective communication	CO3
5	MPI-derived datatypes	CO3
6	Performance evaluation of MPI programs	CO3
7	Performance evaluation of MPI programs	CO3
8	A parallel sorting algorithm.	CO3

Application Areas:

Distributed Computing Systems
Parallel Scientific Computations
Numerical Simulations (e.g., integration, modeling)
Big Data Processing in Clusters
Parallel Sorting in Data Centers

Review Questions / Questions appeared in the previous year (CO):

1	What is MPI? Explain its role in distributed memory programming.
2	Describe the basic MPI functions used for point-to-point communication.
3	How is the trapezoidal rule implemented using MPI for parallel numerical integration?
4	Explain how Input/Output operations are handled in MPI programs.
5	What is collective communication in MPI? Give examples of collective communication functions.
6	What are MPI-derived datatypes? Why are they useful in MPI programming?
7	How do you evaluate the performance of MPI programs in a distributed environment?
8	Describe a simple parallel sorting algorithm using MPI with an example.
9	What is MPI? Explain its role in distributed memory programming.
10	Describe the basic MPI functions used for point-to-point communication.

MODULE – IV

Title:	Shared-memory programming with OpenMP	Appr. Time:	8 Hrs
MO:			Bloom's Level
At the end of the Module, the student will be able to:			
1.Understand OpenMP pragmas and directives for parallel programming.			L3
2.Apply the trapezoidal rule using OpenMP for parallel numerical integration.			L3
3.Distinguish the scope of variables (shared, private) in parallel regions.			L3
4.Use the reduction clause for parallel accumulation operations.			L3



Vivekananda College of Engineering & Technology

[A Unit of Vivekananda Vidyavardhaka Sangha, Puttur ®]

Affiliated to Visvesvaraya Technological University

Approved by AICTE New Delhi & Govt of Karnataka

TCP02

Rev 1.4

CS

21/07/25

COURSE PLAN

5. Identify and address loop-carried dependencies in parallel loops. L3

6. Implement scheduling techniques for workload distribution in loops. L3

Lesson Schedule:

Lecture No.	Portion to be covered	CO
1	Shared-memory programming with OpenMP	CO4
2	openmp pragmas and directives	CO4
3	The trapezoidal rule,	CO4
4	Scope of Variable	CO4
5	The reduction clause, loop carried dependency	CO4
6	scheduling, producers and consumers,	CO4
7	Caches, cache coherence and false sharing in openmp	CO4
8	tasking, tasking, thread safety	CO4

Application Areas:

High-Performance Computing (HPC)

Data Analytics

Real-Time Systems

Image and Signal Processing

Engineering Simulations

Machine Learning Preprocessing

Cache Optimization

Embedded Systems

Review Questions / Questions appeared in the previous year (CO):

1	What are OpenMP pragmas and how are they used in parallel programming?(CO4)
2	How does the reduction clause work in OpenMP and where is it applied?(CO4)
3	Explain loop-carried dependency with an example in OpenMP.(CO4)
4	What is false sharing in OpenMP, and how does it affect performance?(CO4)
5	How does OpenMP handle tasking, and what are its advantages?(CO4)
6	What is cache coherence and why is it important in multi-threaded programming?(CO4)

MODULE – V

Title:	GPU programming with CUDA	Appr. Time:	8 Hrs
MO:			Bloom's
At the end of the Module, the student will be able to:			Level
1. Understand GPUs and General-Purpose GPU (GPGPU) concepts.			L2
2. Explain GPU architectures and their role in parallel processing.			L2
3. Describe heterogeneous computing and its significance.			L2
4. Utilize threads, blocks, and grids in CUDA programming.			L3
5.Understand NVIDIA compute capabilities and device architectures.			L3
Lesson Schedule:			
Lecture No.	Portion to be covered		CO



COURSE PLAN

1	GPUs and GPGPU,	CO5
2	GPU architectures	CO5
3	Heterogeneous computing, Threads, blocks	CO5
4	grids Nvidia compute capabilities and device architectures,	CO5
5	Vector addition, Returning results from CUDA kernels	CO5
6	CUDA trapezoidal rule I,	CO5
7	UDA trapezoidal rule II: improving performance,	CO5
8	CUDA trapezoidal rule III: blocks with more than one warp	CO5

Application Areas:

Machine Learning and Deep Learning
Image and Signal Processing
Real-Time Data Analytics
Engineering Simulations
Financial Modeling
Medical Image

Review Questions / Questions appeared in the previous year (CO):

1	Explain the architecture of a GPU with a neat diagram.(CO5)
2	Differentiate between GPUs and GPGPU.(CO5)
3	Describe the role of threads, blocks, and grids in CUDA programming.(CO5)
4	What are NVIDIA compute capabilities? Explain device architectures.(CO5)
5	Write and explain CUDA code for vector addition.(CO5)
6	Explain the steps to optimize performance using CUDA trapezoidal rule techniques.(CO5)
7	How are results returned from CUDA kernels? Explain with example.(CO5)
8	What is heterogeneous computing? Explain its advantages in CUDA programming.(CO5)

F. LABORATORY CONTENTS

Expt No.	Title of the Experiment	RBT	CO
1.	Write a OpenMP program to sort an array on n elements using both sequential and parallel mergesort(using Section). Record the difference in execution time.	L3	CO 1
2.	Write an OpenMP program that divides the Iterations into chunks containing 2 iterations, respectively (OMP_SCHEDULE=static,2). Its input should be the number	L3	CO 1



COURSE PLAN

	of iterations, and its output should be which iterations of a parallelized for loop are executed by which thread. For example, if there are two threads and four iterations, the output might be the following: a. Thread 0 : Iterations 0 – 1 b. Thread 1 : Iterations 2 – 3		
3.	Write a OpenMP program to calculate n Fibonacci numbers using tasks.	L3	CO 2
4.	Write a OpenMP program to find the prime numbers from 1 to n employing parallel for directive. Record both serial and parallel execution times	L3	C0 2
5.	Write a MPI Program to demonstration of MPI_Send and MPI_Recv	L3	CO 3
6	Write a MPI program to demonstration of deadlock using point to point communication and avoidance of deadlock by altering the call sequence	L3	C0 3
7.	Write a MPI Program to demonstration of Broadcast operation.	L3	C0 4
8.	Write a MPI Program demonstration of MPI_Scatter and MPI_Gather	L3	C0 5
9.	Write a MPI Program to demonstration of MPI_Reduce and MPI_Allreduce (MPI_MAX, MPI_MIN, MPI_SUM, MPI_PROD)	L3	C0 5

EXPERIMENTS

1. EXPERIMENT NO: 1
2. TITLE: OpenMP program to sort an array on n elements using both sequential and parallel Mergesort
3. LEARNING OBJECTIVES: Implement sequential merge-sort Develop parallel mergesort using OpenMP Compare sequential vs parallel performance
4. AIM: To implement a OpenMP program to sort an array on n elements using both sequential and parallel mergesort.
5.Theory: Mergesort is a classic divide-and-conquer sorting algorithm that recursively divides an array into two halves, sorts them individually, and then merges the sorted halves to produce the final sorted array. In a sequential mergesort , this process is performed using a single thread, which limits performance to a single CPU core. To enhance performance, parallel mergesort can be implemented using OpenMP , where the independent sorting of left and right halves can be executed simultaneously using multiple threads. OpenMP provides the #pragma omp parallel



COURSE PLAN

sections directive, allowing different code sections to run concurrently, with each recursive call placed inside separate `#pragma omp` section blocks. This allows both halves of the array to be sorted in parallel, significantly reducing the overall execution time on multi-core systems. The **execution time** of both sequential and parallel versions can be recorded using the `omp_get_wtime()` function to compare performance improvements. While parallel mergesort effectively utilizes CPU resources and accelerates sorting tasks, care must be taken to manage thread creation overhead for small subarrays and to handle shared data correctly to avoid race conditions. Overall, parallel mergesort using OpenMP sections offers a practical approach to demonstrate the benefits of parallel programming in sorting operations.

6. Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define SIZE 100000 // Adjust size as needed

// Function to merge two sorted halves
void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int *L = (int*)malloc(n1 * sizeof(int));
    int *R = (int*)malloc(n2 * sizeof(int));

    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;

    while (i < n1 && j < n2)
        arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];

    while (i < n1)
        arr[k++] = L[i++];
    while (j < n2)
        arr[k++] = R[j++];

    free(L);
    free(R);
}
```



COURSE PLAN

```
// Sequential Mergesort
void sequential_mergesort(int arr[], int left, int right) {
    if (left < right) {
        int mid = (left + right) / 2;
        sequential_mergesort(arr, left, mid);
        sequential_mergesort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}
```

```
// Parallel Mergesort using OpenMP Sections
void parallel_mergesort(int arr[], int left, int right) {
    if (left < right) {
        int mid = (left + right) / 2;

        #pragma omp parallel sections
        {
            #pragma omp section
            parallel_mergesort(arr, left, mid);

            #pragma omp section
            parallel_mergesort(arr, mid + 1, right);
        }

        merge(arr, left, mid, right);
    }
}
```

```
int main() {
    int arr_seq[SIZE], arr_par[SIZE];
    double start, end;

    // Initialize random array
    for (int i = 0; i < SIZE; i++) {
        int val = rand();
        arr_seq[i] = val;
        arr_par[i] = val;
    }

    // Sequential Mergesort Timing
    start = omp_get_wtime();
    sequential_mergesort(arr_seq, 0, SIZE - 1);
```



COURSE PLAN

```
end = omp_get_wtime();  
printf("Sequential Time: %f seconds\n", end - start);
```

```
// Parallel Mergesort Timing  
start = omp_get_wtime();  
parallel_mergesort(arr_par, 0, SIZE - 1);  
end = omp_get_wtime();  
printf("Parallel Time: %f seconds\n", end - start);
```

```
return 0;
```

```
}
```

7. Block diagram:

8. Result and Conclusion:

Compilation Command:

```
gcc mergesort_openmp.c -fopenmp -o mergesort  
./mergesort
```

OUTPUT:

Sequential Time: 0.257134 seconds

Parallel Time: 0.114289 seconds

9. Learning Outcome:

- Understand the concept of merge-sort as a divide-and-conquer algorithm.
- Implement both sequential and parallel merge-sort using OpenMP sections.
- Use OpenMP directives to parallelize recursive functions effectively.
- Measure and compare execution time between sequential and parallel merge-sort.

10. Application Area

Data Analytics

Machine Learning Pre-processing

11. Remarks

1. EXPERIMENT NO: 2

2. TITLE: **Thread**

3. LEARNING OBJECTIVES:

To explain the concept of **parallel for-loops** using OpenMP.

To illustrate the use of **static scheduling** with a specified **chunk size**.



COURSE PLAN

To demonstrate how to divide loop iterations among multiple threads.

To show the use of **OpenMP environment variables** for controlling scheduling policies.

4. AIM: Write an OpenMP program that divides the Iterations into chunks containing 2 iterations, respectively (OMP_SCHEDULE=static,2). Its input should be the number of iterations, and its output should be which iterations of a parallelized for loop are executed by which thread.

For example, if there are two threads and four iterations, the output might be the following:

- Thread 0 : Iterations 0 — 1
- Thread 1 : Iterations 2 – 3

5. Theory:

This program demonstrates how **OpenMP static scheduling** with a **chunk size of 2** divides iterations among available threads in a parallelized for-loop. In parallel computing, distributing work efficiently among threads is critical for achieving optimal performance. OpenMP provides several scheduling strategies, and **static scheduling** is one of the simplest and most predictable.

In **static scheduling**, the iterations of a loop are divided into fixed-size chunks and assigned to threads in a round-robin manner at compile or run-time. In this program, each thread receives **2 consecutive iterations** as one chunk (static,2). OpenMP's `omp_set_schedule()` function sets this scheduling strategy programmatically, while the `schedule(runtime)` clause in the `omp for` directive allows the schedule to be controlled via environment variables (OMP_SCHEDULE).

During execution:

- The total number of iterations is input by the user.
- Each thread executes its assigned iterations, and outputs which iterations it handled using `omp_get_thread_num()` to identify the thread.
- The environment variable `OMP_NUM_THREADS` controls the number of threads participating in the parallel region.

This approach helps visualize load distribution among threads and teaches how **chunk size** affects parallel workload distribution. Such static scheduling is particularly useful when loop iterations take approximately equal time to execute.

In summary, the program showcases basic **parallel for-loop management** in OpenMP using **static scheduling with controlled chunk size**, highlighting how iterations are distributed across multiple threads to improve parallel execution efficiency.

6. Program:

```
#include <stdio.h>
#include <omp.h>
```

```
int main() {
```



COURSE PLAN

```
int num_iterations;
printf("Enter number of iterations: ");
scanf("%d", &num_iterations);

// Set static schedule with chunk size 2
omp_set_schedule(omp_sched_static, 2);

#pragma omp parallel
{
    int thread_id = omp_get_thread_num();

    #pragma omp for schedule(runtime)
    for (int i = 0; i < num_iterations; i++) {
        printf("Thread %d : Iteration %d\n", thread_id, i);
    }
}

return 0;
}
```

7.Block diagram:

8.Result and Conclusion:

Compilation Command:

```
gcc -fopenmp schedule_static.c -o schedule_static
export OMP_NUM_THREADS=2
export OMP_SCHEDULE="static,2"
./schedule_static
```

OUTPUT:

```
Enter number of iterations: 4
Thread 0 : Iteration 0
Thread 0 : Iteration 1
Thread 1 : Iteration 2
Thread 1 : Iteration 3
```

9.Learning Outcome:

Understand the concept of static scheduling in OpenMP.

Use chunk size to control workload distribution in parallel loops.

Apply OpenMP environment variables (OMP_NUM_THREADS, OMP_SCHEDULE).

Implement parallel for-loops using #pragma omp for directive.

10.Application Area

Data Analytics

Machine Learning Preprocessing

11.Remarks



COURSE PLAN

1. EXPERIMENT NO: 3

2. TITLE: **Fibonacci numbers using tasks**

3. LEARNING OBJECTIVES:

Understand static scheduling with chunk size in OpenMP

Implement parallel for-loop using OpenMP

Distribute loop iterations among threads

Use OpenMP environment variables for scheduling control

Visualize thread-wise execution of iterations

Analyze impact of chunk size on performance

4. AIM: Write a OpenMP program to calculate n Fibonacci numbers using tasks.

5. Theory

In parallel computing, efficiently distributing loop iterations among threads is critical for performance optimization. OpenMP provides several scheduling strategies to manage this distribution, and **static scheduling** is one of the simplest and most predictable. In **static scheduling with chunk size**, the compiler divides the total iterations into fixed-size chunks and assigns them to threads in a round-robin fashion. For example, with a chunk size of 2 (static,2), every thread receives two consecutive iterations at a time.

This program uses **OpenMP's parallel for-loop** to divide a user-defined number of iterations among multiple threads using **static scheduling** with a **chunk size of 2**. The schedule is set either using the **omp_set_schedule()** function in the code or via the **OMP_SCHEDULE environment variable**, using the **schedule(runtime)** clause in the OpenMP directive to read that runtime setting.

Each thread identifies itself using **omp_get_thread_num()**, and prints out the iterations it executes. This allows visualization of how the work is distributed across threads during parallel execution. Using the **OMP_NUM_THREADS** environment variable, users can control how many threads participate in the computation.

This example helps understand basic load distribution concepts in OpenMP and shows how the chunk size affects the assignment of work and the utilization of threads in a parallel region.

6. Program

```
#include <stdio.h>
```

```
#include <omp.h>
```



COURSE PLAN

// Recursive Fibonacci using OpenMP tasks

```
int fibonacci(int n) {
    int x, y;

    if (n <= 1)
        return n;

    #pragma omp task shared(x)
    x = fibonacci(n - 1);

    #pragma omp task shared(y)
    y = fibonacci(n - 2);

    #pragma omp taskwait
    return x + y;
}

int main() {
    int n, result;

    printf("Enter value of n (Fibonacci number to calculate): ");
    scanf("%d", &n);

    double start_time = omp_get_wtime();

    #pragma omp parallel
    {
        #pragma omp single
        result = fibonacci(n);
    }

    double end_time = omp_get_wtime();

    printf("Fibonacci number %d is: %d\n", n, result);
    printf("Time taken: %f seconds\n", end_time - start_time);

    return 0;
}
```

7.Block diagram:

8.Result and Conclusion:

Compilation Command:

```
gcc -fopenmp fibonacci_task.c -o fibonacci_task
./fibonacci_task
```



COURSE PLAN

OUTPUT:

Enter value of n (Fibonacci number to calculate): 10

Fibonacci number 10 is: 55

Time taken: 0.002134 seconds

9.Learning Outcome:

Understand OpenMP task parallelism

Implement recursive Fibonacci using tasks

Use task and task wait directives

Apply single construct for task initiation

Measure execution time of parallel tasks

Analyze task-based performance benefits

10.Application Area

Data Analytics

Machine Learning Pre-processing

11.Remarks

1. EXPERIMENT NO: 4

2. TITLE: **Prime number**

3. LEARNING OBJECTIVES:

Implement sequential merge-sort

Develop parallel mergesort using OpenMP

Compare sequential vs parallel performance

4. AIM: Write a OpenMP program to find the prime numbers from 1 to n employing parallel for directive. Record both serial and parallel execution times..

5.Theory

Inding prime numbers involves checking each number for divisibility from 2 to $\sqrt{n}$.

In serial execution, each number is checked sequentially using a single thread.

In parallel execution, the workload (numbers to check) is divided among threads using the **#pragma omp parallel for** directive.

A **critical section** ensures that multiple threads do not print simultaneously (avoiding mixed



COURSE PLAN

output).

omp_get_wtime() measures and compares serial vs parallel execution times to analyze speedup.

6. Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

// Function to check if a number is prime
int is_prime(int num) {
    if (num <= 1)
        return 0;
    for (int i = 2; i * i <= num; i++) {
        if (num % i == 0)
            return 0;
    }
    return 1;
}

int main() {
    int n;
    printf("Enter the value of n: ");
    scanf("%d", &n);

    double start, end;

    // Serial Execution
    start = omp_get_wtime();
    printf("Serial Prime Numbers:\n");
    for (int i = 1; i <= n; i++) {
        if (is_prime(i))
            printf("%d ", i);
    }
    printf("\n");
    end = omp_get_wtime();
    printf("Serial Execution Time: %f seconds\n\n", end - start);

    // Parallel Execution
    start = omp_get_wtime();
    printf("Parallel Prime Numbers:\n");

    #pragma omp parallel for schedule(dynamic)
```



COURSE PLAN

```
for (int i = 1; i <= n; i++) {  
    if (is_prime(i)) {  
        #pragma omp critical  
        {  
            printf("%d ", i);  
        }  
    }  
}  
printf("\n");  
end = omp_get_wtime();  
printf("Parallel Execution Time: %f seconds\n", end - start);  
  
return 0;  
}
```

7.Block diagram:

8.Result and Conclusion:

Compilation Command:

```
gcc prime_openmp.c -fopenmp -o prime_openmp  
./prime_openmp
```

Enter the value of n: 20

Serial Prime Numbers:

2 3 5 7 11 13 17 19

Serial Execution Time: 0.000045 seconds

Parallel Prime Numbers:

2 3 5 7 11 13 17 19

Parallel Execution Time: 0.000031 seconds

9.Learning Outcome:

Understand parallel loop execution using OpenMP

Implement prime number generation using parallel for directive

Use critical sections for safe output in multi-threaded programs

Measure and compare serial vs parallel execution times

Analyze performance improvement with parallel execution

10.Application Area



COURSE PLAN

Data Analytics

Machine Learning Pre-processing

11. Remarks

1. EXPERIMENT NO: 5

2. TITLE: **PI_Send and MPI_Recv**

3. LEARNING OBJECTIVES:

To explain the concept of **message passing** in MPI using send and receive operations.

To demonstrate the use of **MPI_Send** and **MPI_Recv** functions for data exchange between processes.

To help students understand how **process ranks** control communication in distributed applications.

To enable students to implement simple **point-to-point communication** using MPI.

To illustrate practical execution of MPI programs in a **distributed memory environment**.

4. AIM: Write a MPI Program to demonstration of MPI_Send and MPI_Recv

5. Theory: **Message Passing Interface (MPI)** is a standardized and portable API used for parallel programming in distributed memory systems. **Point-to-point communication** in MPI allows individual processes to send and receive messages directly from one another, which is essential for data sharing across processes.

Two fundamental MPI functions for point-to-point communication are **MPI_Send** and **MPI_Recv**:

- **MPI_Send**: This function is used by a process to send data to another process. The sender specifies the data buffer, count, datatype, destination process rank, message tag, and communicator.
- **MPI_Recv**: This function is used by a process to receive data sent by another process. The receiver specifies the buffer to store the data, source process rank, tag, communicator, and status.

In a typical example:

- **Process 0** sends a message (data) using MPI_Send.
- **Process 1** receives that message using MPI_Recv.

Every MPI process is identified by a **rank**, assigned within the communicator group (usually MPI_COMM_WORLD). Communication occurs between specific ranks.

```
#include <stdio.h>
```

```
#include <mpi.h>
```

```
int main(int argc, char *argv[]) {
```



COURSE PLAN

```
int rank, size;
```

```
int number;
```

```
MPI_Init(&argc, &argv);          // Initialize MPI environment
```

```
MPI_Comm_rank(MPI_COMM_WORLD, &rank); // Get current process rank
```

```
MPI_Comm_size(MPI_COMM_WORLD, &size); // Get total number of processes
```

```
if (size < 2) {
```

```
    if (rank == 0)
```

```
        printf("Please run with at least 2 processes.\n");
```

```
    MPI_Finalize();
```

```
    return 0;
```

```
}
```

```
if (rank == 0) {
```

```
    number = 100; // Message to send
```

```
    MPI_Send(&number, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
```

```
    printf("Process 0 sent number %d to Process 1.\n", number);
```

```
}
```

```
if (rank == 1) {
```

```
    MPI_Recv(&number,          1,          MPI_INT,          0,          0,
MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

```
    printf("Process 1 received number %d from Process 0.\n", number);
```

```
MPI_Finalize();
```

```
return 0;
```

```
}
```

6. Block diagram:

8. Result and Conclusion:

Compilation Command:

```
mpicc mpi_send_recv.c -o mpi_send_recv
```

```
mpirun -np 2 ./mpi_send_recv
```

OUTPUT:

Process 0 sent number 100 to Process 1.

Process 1 received number 100 from Process 0.

- Understand **point-to-point communication** in MPI.
- Learn to use **MPI_Send** and **MPI_Recv** functions for message passing.
- Implement basic **process-to-process communication** using MPI.
- Understand the role of **process ranks** in distributed systems.
- Gain practical experience in writing and executing **MPI programs**.



COURSE PLAN

10.Application Area

Data Analytics

Machine Learning Pre-processing

11.Remarks

1. EXPERIMENT NO: 6

2. TITLE: **Deadlock**

3. LEARNING OBJECTIVES:

To explain the concept of **deadlock** in MPI communication.

To demonstrate how **MPI_Send** and **MPI_Recv** can lead to deadlock if misused.

To help students implement safe **point-to-point communication** using MPI.

To encourage analysis of **communication patterns** in distributed applications.

4. AIM: Write a MPI program to demonstration of deadlock using point to point communication and avoidance of deadlock by altering the call sequence .

5.Theory:

Two processes attempt to **send and receive messages simultaneously**, which can lead to a **deadlock** if not handled carefully.The **deadlock version** sends first from both processes, causing both to wait indefinitely.The **resolved version** alters the call sequence to avoid deadlock.

6.Program:

```
#include <stdio.h>
```

```
#include <mpi.h>
```

```
int main(int argc, char* argv[]) {
```

```
    int rank, data = 100;
```

```
    MPI_Init(&argc, &argv);
```

```
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
```

```
    if (rank == 0) {
```

```
        MPI_Send(&data, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
```

```
        MPI_Recv(&data, 1, MPI_INT, 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

```
        printf("Process 0 completed communication.\n");
```

```
    } else if (rank == 1) {
```

```
        MPI_Send(&data, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
```

```
        MPI_Recv(&data, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

```
        printf("Process 1 completed communication.\n");
```



COURSE PLAN

```
}

MPI_Finalize();
return 0;
}

#include <stdio.h>
#include <mpi.h>

int main(int argc, char* argv[]) {
    int rank, data = 200;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if (rank == 0) {
        MPI_Send(&data, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
        MPI_Recv(&data, 1, MPI_INT, 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Process 0 completed communication.\n");
    } else if (rank == 1) {
        MPI_Recv(&data, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        MPI_Send(&data, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
        printf("Process 1 completed communication.\n");
    }

    MPI_Finalize();
    return 0;
}
```

11. Block diagram:

12. Result and Conclusion:

Compilation Command:

```
mpicc mpi_deadlock.c -o mpi_deadlock
```

```
mpirun -np 2 ./mpi_deadlock
```

OUTPUT:

(Process 0 or Process 1 may send data...)

13. Learning Outcome:

Understand the concept of **deadlock** in MPI point-to-point communication.

Identify why **improper send-receive ordering** can cause deadlock.

Learn to implement **MPI_Send** and **MPI_Recv** correctly to avoid deadlock.

Gain practical experience in writing both **deadlock-prone** and **deadlock-free** MPI programs.

Analyze how altering the **call sequence** prevents communication blocking.

Develop skills in managing **safe communication patterns** in distributed systems.



COURSE PLAN

14. Application Area

Data Analytics

Machine Learning Pre-processing

12. Remarks

1. EXPERIMENT NO: 7

2. TITLE: **Broadcast operation**

3. LEARNING OBJECTIVES:

Understand the concept of **deadlock** in MPI point-to-point communication.

Learn the correct use of **MPI_Send** and **MPI_Recv** to avoid deadlock.

Demonstrate how altering **send-receive sequence** prevents deadlock.

Develop the ability to write **safe and efficient MPI programs**.

Analyze communication flow to ensure **deadlock-free execution**.

4. AIM: Write a MPI Program to demonstration of Broadcast operation..

5. Theory:

In **parallel and distributed computing**, efficient communication between processes is essential. The **broadcast operation** is a form of collective communication in MPI, where **one process (root)** sends the same piece of data to **all other processes** in a communicator.

MPI provides the **MPI_Bcast** function to perform this operation. Unlike point-to-point communication (using **MPI_Send** and **MPI_Recv**), **MPI_Bcast** allows a single message to be automatically distributed to all processes without explicitly coding multiple sends and receives.

6. Program:

```
#include <stdio.h>
```

```
#include <mpi.h>
```

```
int main(int argc, char* argv[]) {  
    int rank, size, data;
```

```
    MPI_Init(&argc, &argv);           // Initialize MPI environment
```

```
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);    // Get current process rank
```

```
    MPI_Comm_size(MPI_COMM_WORLD, &size);    // Get total number of processes
```

```
    if (rank == 0) {  
        printf("Enter a value to broadcast: ");  
        scanf("%d", &data);
```



COURSE PLAN

}

// Broadcast data from process 0 to all other processes

MPI_Bcast(&data, 1, MPI_INT, 0, MPI_COMM_WORLD);

printf("Process %d received data = %d\n", rank, data);

MPI_Finalize();

return 0;

}

7.Block diagram:

8.Result and Conclusion:

Compilation Command:

gcc broadcast_openmp.c -fopenmp -o broadcast
./broadcast

OUTPUT:

Enter a value to broadcast: 50

Process 0 received data = 50

Process 1 received data = 50

Process 2 received data = 50

Process 3 received data = 5

9.Learning Outcome:

Understand the concept of **broadcast communication** in MPI.

Learn to use the **MPI_Bcast** function for one-to-many data distribution.

Implement collective communication in **distributed memory systems**.

Observe how data is efficiently shared from a **single source process to all other processes**.

Gain practical experience in using **MPI collective operations**.

10.Application Area

Data Analytics

Machine Learning Pre-processing

11.Remarks

1. EXPERIMENT NO: 8

2. TITLE: **MPI_Scatter and MPI_Gather**

3. LEARNING OBJECTIVES:

Understand the concept of collective communication in MPI.



COURSE PLAN

Learn how to distribute data using MPI_Scatter.
Learn how to collect data using MPI_Gather.
Write MPI programs using parallel processing techniques.
Perform independent computation on distributed data.
Practice synchronization and coordination among processes.

4.AIM: Write a MPI Program demonstration of MPI_Scatter and MPI_Gather

5.Theory:

What is MPI?

MPI (Message Passing Interface) is a standardized and portable message-passing system designed to allow processes to communicate with one another in a distributed memory system. It is widely used for parallel computing on clusters and supercomputers.

♦ Collective Communication in MPI

MPI provides **collective communication operations** that involve all processes in a communicator. These include:

- Broadcast
- Scatter
- Gather
- Allgather
- Reduce
- Allreduce

Two important collective operations are **MPI_Scatter** and **MPI_Gather**.

♦ MPI_Scatter

MPI_Scatter is used to **distribute chunks of data** from the root process to all processes in a communicator

```
6.Program#include <mpi.h>
#include <stdio.h>
```

```
int main(int argc, char* argv[]) {
    int rank, size;
    int data[8], recv_data;
    int gathered_data[8];
```

```
    MPI_Init(&argc, &argv);           // Initialize MPI
    MPI_Comm_rank(MPI_COMM_WORLD, &rank); // Get process rank
```



COURSE PLAN

```
MPI_Comm_size(MPI_COMM_WORLD, &size); // Get total number of processes

if (size != 8) {
    if (rank == 0) {
        printf("Please run the program with 8 processes.\n");
    }
    MPI_Finalize();
    return 0;
}

// Initialize the array in the root process
if (rank == 0) {
    for (int i = 0; i < 8; i++) {
        data[i] = i + 1; // data = {1, 2, 3, 4, 5, 6, 7, 8}
    }
}

// Scatter: send one integer to each process
MPI_Scatter(data, 1, MPI_INT, &recv_data, 1, MPI_INT, 0, MPI_COMM_WORLD);

// Each process multiplies its received value by 10
recv_data *= 10;

// Gather: collect the modified data at the root process
MPI_Gather(&recv_data, 1, MPI_INT, gathered_data, 1, MPI_INT,
0, MPI_COMM_WORLD);

// Display result at root
if (rank == 0) {
    printf("Gathered data after processing:\n");
    for (int i = 0; i < 8; i++) {
        printf("%d ", gathered_data[i]);
    }
    printf("\n");
}

MPI_Finalize(); // Finalize MPI
return 0;
}
```

7.Block diagram:

8.Result and Conclusion:

Compilation Command:

mpirun -np 8 ./scatter_gather



COURSE PLAN

mpiexec -n 8 ./scatter_gather

OUTPUT:

Gathered data after processing:

10 20 30 40 50 60 70 80

1. Learning Outcome:

Understand and explain MPI_Scatter and MPI_Gather.

Write and implement MPI programs using collective communication.

Perform parallel data distribution and collection.

Analyze the efficiency of collective operations in MPI.

Compile and execute MPI programs using command-line tools.

Apply learned concepts in real-world parallel applications.

2. Application Area

Computational Fluid Dynamics (CFD)

Data Analytics

Machine Learning Pre-processing

13. Remarks

1. EXPERIMENT NO: 9

2. TITLE: **MPI_Reduce and MPI_Allreduce (MPI_MAX, MPI_MIN, MPI_SUM, MPI_PROD).**

3. LEARNING OBJECTIVES:

Understand MPI_Reduce and MPI_Allreduce operations and their differences.

Explore reduction operations like MPI_SUM, MPI_PROD, MPI_MIN, and MPI_MAX.

Implement collective communication functions in an MPI environment.

Demonstrate distributed aggregation of data using reduce techniques.

Write parallel programs that collect and compute results in both centralized (Reduce) and distributed (Allreduce) ways.

Apply MPI reduction operations in real-world problems such as parallel statistics, scientific computations, and matrix operations.

4. AIM: Write a MPI Program to demonstration of MPI_Reduce and MPI_Allreduce (MPI_MAX, MPI_MIN, MPI_SUM, MPI_PROD).

5.Theory:

Mergesort is a classic divide-and-conquer sorting algorithm that recursively divides an array into two halves, sorts them individually, and then merges the sorted halves to produce the final sorted array. In a **sequential mergesort**, this process is performed using a single thread, which limits performance to a single CPU core. To enhance performance, **parallel mergesort** can be implemented using **OpenMP**, where the independent sorting of left and right halves can be executed simultaneously using multiple threads. OpenMP provides the `#pragma omp parallel`



COURSE PLAN

sections directive, allowing different code sections to run concurrently, with each recursive call placed inside separate `#pragma omp` section blocks. This allows both halves of the array to be sorted in parallel, significantly reducing the overall execution time on multi-core systems. The **execution time** of both sequential and parallel versions can be recorded using the `omp_get_wtime()` function to compare performance improvements. While parallel mergesort effectively utilizes CPU resources and accelerates sorting tasks, care must be taken to manage thread creation overhead for small subarrays and to handle shared data correctly to avoid race conditions. Overall, parallel mergesort using OpenMP sections offers a practical approach to demonstrate the benefits of parallel programming in sorting operations.

6.Program:

```
#include <mpi.h>
```

```
#include <stdio.h>
```

```
int main(int argc, char* argv[]) {
```

```
    int rank, size;
```

```
    int value;
```

```
    int reduce_sum, reduce_prod, reduce_min, reduce_max;
```

```
    int allreduce_sum, allreduce_prod, allreduce_min, allreduce_max;
```

```
    MPI_Init(&argc, &argv);           // Initialize the MPI environment
```

```
    MPI_Comm_rank(MPI_COMM_WORLD, &rank); // Get the rank of the process
```

```
    MPI_Comm_size(MPI_COMM_WORLD, &size); // Get the total number of processes
```

```
    // Each process uses its rank + 1 as its value
```

```
    value = rank + 1;
```

```
    // Perform MPI_Reduce operations: result only available at root (rank 0)
```

```
    MPI_Reduce(&value, &reduce_sum, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
```

```
    MPI_Reduce(&value, &reduce_prod, 1, MPI_INT, MPI_PROD, 0, MPI_COMM_WORLD);
```

```
    MPI_Reduce(&value, &reduce_min, 1, MPI_INT, MPI_MIN, 0, MPI_COMM_WORLD);
```

```
    MPI_Reduce(&value, &reduce_max, 1, MPI_INT, MPI_MAX, 0, MPI_COMM_WORLD);
```

```
    // Print MPI_Reduce results at root
```

```
    if (rank == 0) {
```

```
        printf("MPI_Reduce results (at root process):\n");
```

```
        printf("SUM = %d\n", reduce_sum);
```

```
        printf("PROD = %d\n", reduce_prod);
```

```
        printf("MIN = %d\n", reduce_min);
```

```
        printf("MAX = %d\n", reduce_max);
```

```
        printf("\n");
```

```
    }
```

```
    // Perform MPI_Allreduce operations: result available at all processes
```



COURSE PLAN

```
MPI_Allreduce(&value, &allreduce_sum, 1, MPI_INT, MPI_SUM, MPI_COMM_WORLD);
MPI_Allreduce(&value, &allreduce_prod, 1, MPI_INT, MPI_PROD, MPI_COMM_WORLD);
MPI_Allreduce(&value, &allreduce_min, 1, MPI_INT, MPI_MIN, MPI_COMM_WORLD);
MPI_Allreduce(&value, &allreduce_max, 1, MPI_INT, MPI_MAX, MPI_COMM_WORLD);

// Print MPI_Allreduce results from all processes
printf("Process %d: MPI_Allreduce results:\n", rank);
printf("SUM = %d, PROD = %d, MIN = %d, MAX = %d\n\n",
    allreduce_sum, allreduce_prod, allreduce_min, allreduce_max);

MPI_Finalize(); // Finalize the MPI environment
return 0;
}
```

7.Block diagram:

8.Result and Conclusion:

Compilation Command:

```
mpicc reduce_allreduce.c -o reduce_allreduce
mpirun -np 4 ./reduce_allreduce
mpiexec -n 4 ./reduce_allreduce
```

OUTPUT:

MPI_Reduce results (at root process):

SUM = 10

PROD = 24

MIN = 1

MAX = 4

Process 0: MPI_Allreduce results:

SUM = 10, PROD = 24, MIN = 1, MAX = 4

Process 1: MPI_Allreduce results:

SUM = 10, PROD = 24, MIN = 1, MAX = 4

Process 2: MPI_Allreduce results:

SUM = 10, PROD = 24, MIN = 1, MAX = 4

Process 3: MPI_Allreduce results:

SUM = 10, PROD = 24, MIN = 1, MAX = 4

9.Learning Outcome:

Explain the functionality and purpose of MPI_Reduce and MPI_Allreduce in collective communication.

Differentiate between reduction and all-reduction operations in terms of result distribution and usage.



COURSE PLAN

Use predefined reduction operators like MPI_SUM, MPI_PROD, MPI_MIN, and MPI_MAX effectively in MPI programs. Write and execute parallel programs that aggregate distributed data using reduction operations. Demonstrate parallel aggregation techniques in solving computational problems across multiple processes.
10.Application Area Data Analytics Machine Learning Pre-processing
11.Remarks -----

F.INTERNAL ASSESSMENT TEST MODEL QUESTION PAPER (from Modules 1 & 2)

Dept: CS		Sem / Div:7	Course: Parallel Computing	Course Code: BCS702	
Date: dd/mm/yyyy		Time: 60 Min.	Max Marks: 50	Elective: N	
Note: Answer any 2 full questions, choosing one full question from each part.					
QN		Questions	Marks	RBT	CO
PART A					
1	a	Explain GPU architecture with a neat diagram.	10	L2	CO1
	b	Differentiate between GPUs and GPGPU.	10	L2	CO1
	c	Describe heterogeneous computing with example.	5	L2	CO1
OR					
2	a	Explain the structure of threads, blocks, and grids in CUDA with example.	10	L3	CO1
	b	Write a CUDA program for vector addition.	10	L3	CO1
	c	Demonstrate returning results from CUDA kernels with code snippet.	5	L3	CO1
PART B					
3	a	Explain how hybrid systems improve computational performance.	10	L2	CO2
	b	Explain speedup and efficiency in MIMD systems	10	L2	CO2
	c	What is Amdahl's Law? List the factors affecting GPU performance	5	L2	CO2
OR					
4	a	Explain scalability in MIMD architectures.	10	L2	CO2
	b	Explain a simple parallel application using MIMD concepts and explain	10	L3	CO2



Vivekananda College of Engineering & Technology

[A Unit of Vivekananda Vidyavardhaka Sangha, Puttur ®]

Affiliated to Visvesvaraya Technological University

Approved by AICTE New Delhi & Govt of Karnataka

TCP02

Rev 1.4

CS

21/07/25

COURSE PLAN

	performance measurements.			
c	Explain the steps to take timing measurements of MIMD programs.	5	L2	C02

G. CONTINUOUS INTERNAL EVALUATION

Evaluation	Weightage in Marks	Scaled Down
IA Test – 1	50	25
IA Test – 2	50	
Marks for Different Components: Additional Assessment Tools (AATs) – (Assignments/ Open Book Test/ Written Quiz/ Seminar/ Report Writing/ Conduction of Experiments/ Micro-Project)	10+15	25
Total Marks		50